

Sri Lanka Institute of Information Technology

IT3060 - Human Computer Interaction



Lab Sheet 05
IT23658790
B D Wanigatunga

Introduction

This lab report responds to Lab Exercise 05 for IT3060 – Human Computer Interaction, building directly on the FitFlow case study. Having already completed user research, synthesis, ideation, wireframing and usability testing, the FitFlow team must now select the technology stack that will carry the validated designs into a working, deployable product. The activities below compare cross-platform frontend frameworks, backend/database/authentication options, consolidate the findings into a weighted decision matrix, propose a high-level system architecture with an Architecture Decision Record (ADR), and set out the GitHub repository structure used to manage the project deliverables.

All comparisons are evaluated specifically against FitFlow's requirements identified in the case study: an AI-powered personalized workout engine, private social circles, camera-based nutrition logging using computer vision, offline-capable and accessible interfaces, strong data privacy (GDPR/CCPA), and the need to move quickly with a mid-sized team.

Activity 1: Comparing Flutter, React Native, Kotlin Multiplatform and Swift/SwiftUI

Four cross-platform and native approaches were evaluated as candidates for the FitFlow mobile frontend: Flutter (Dart), React Native (JavaScript/TypeScript), Kotlin Multiplatform Mobile (KMM), and native Swift/SwiftUI (iOS-only).

1.1 Comparison Table

Criterion	Flutter	React Native	Kotlin Multiplatform	Swift/SwiftUI
Development speed	Fast – single codebase, hot reload	Fast – single codebase, huge component ecosystem	Moderate – shared business logic only, UI still per-platform	Slow for cross-platform – iOS only, Android needs a separate codebase
Code reusability	Very high (~95%+ incl. UI)	Very high (~85–95%, native modules for edge cases)	Moderate (~40–60%, logic layer only)	None across platforms (native)
Performance	Near-native (compiled, own rendering engine – Skia/Impeller)	Near-native (bridges to native components, occasional JS-thread bottlenecks)	Native (compiles to platform binaries)	Best-in-class native performance
Ecosystem support	Strong and growing, Google-backed	Very mature, largest cross-platform community, Meta-backed	Growing but smaller, JetBrains-backed	Mature but Apple-only

Criterion	Flutter	React Native	Kotlin Multiplatform	Swift/SwiftUI
Learning curve	Moderate – new language (Dart)	Gentle for teams that already know JS/React	Steep – requires Kotlin plus native UI (SwiftUI/Jetpack Compose)	Steep for non-Apple teams; easy for existing iOS developers
Web compatibility	Good (Flutter Web, same codebase)	Good via React Native Web	Limited (Compose Multiplatform Web is early-stage)	None (no web target)
AI/ML integration	TensorFlow Lite plugin, ML Kit bindings available	Strong – TensorFlow Lite, ML Kit, many community bridges	Native ML Kit / Core ML access via shared logic	Best native access to Core ML
Real-time features	Good, via platform channels for sockets/push	Excellent – mature libraries for WebSockets, Firebase, push	Good, native APIs directly accessible	Excellent (native)
Maintenance cost	Low – one codebase for both platforms	Low – one codebase, large hiring pool	Medium – UI still duplicated per platform	High – separate Android codebase still required
Security	Good, relies on platform security primitives	Good, mature libraries for secure storage/auth	Good, closer to native security APIs	Excellent, tightest OS-level integration

Table 1.1 – Frontend framework comparison across FitFlow-relevant criteria

1.2 Strengths and Weaknesses Summary

Flutter

- Strengths: single codebase for mobile and web, consistent pixel-perfect UI, strong animation support well suited to fitness/progress visualizations.
- Weaknesses: smaller pool of experienced Dart developers; larger app binary size; some native SDKs (e.g. certain fitness wearables) require custom plugin work.

React Native

- Strengths: enormous ecosystem, easiest hiring given prevalence of JavaScript/TypeScript skills, first-class Firebase and TensorFlow Lite support, React Native Web extends reach to a browser client with modest extra effort.
- Weaknesses: JavaScript bridge can bottleneck highly complex animations; occasional native-module version fragmentation.

Kotlin Multiplatform (KMM)

- Strengths: shares only business logic (networking, data models, AI inference calls) while keeping fully native UI, giving the best possible native feel.
- Weaknesses: UI layer is still built twice (SwiftUI + Jetpack Compose), which roughly doubles frontend engineering effort for a mid-sized team; immature web story.

Swift/SwiftUI

- Strengths: unmatched iOS performance, tightest integration with HealthKit/Core ML – valuable for a fitness app.
- Weaknesses: iOS-only; would require a fully separate Android codebase (e.g. Kotlin/Jetpack Compose), roughly doubling both build and long-term maintenance cost; no web path at all.

1.3 Suitability for FitFlow

FitFlow requires a seamless iOS/Android/web experience, rapid delivery from a mid-sized team, rich animation for progress and AI recommendation cards, and reliable access to on-device ML (TensorFlow Lite) and camera-based computer vision. Native-only options (Swift/SwiftUI, and to a lesser extent KMM) fail the web-compatibility and single-team-velocity requirements, since they either double the UI engineering effort or omit the web target entirely. Flutter is a strong contender but React Native's larger ecosystem, easier hiring, and proven track record integrating Firebase (already selected for real-time social and notification features) give it an edge for a startup that must move quickly.

1.4 Recommendation

Recommended option: React Native (with React Native Web for the optional browser client).

Justification: React Native offers the best balance of development speed, code reusability (~85–95%), a mature AI/ML and Firebase integration story, and the largest available talent pool – all critical for a mid-sized startup team working against a competitive timeline. Flutter remains a credible fallback if the team later prioritizes pixel-perfect custom UI over ecosystem breadth; a KMM or fully-native approach would only be justified if FitFlow pivoted to prioritize best-in-class native performance over time-to-market, which the case study's emphasis on speed and cost-effectiveness does not support.

Activity 2: Backend, Database and Authentication Options

2.1 Backend Frameworks

Criterion	Node.js / Express / NestJS	Python / FastAPI	Go (Gin/Fiber)
Development speed	Fast – shares JavaScript/TypeScript with the React Native frontend	Fast – concise syntax, automatic OpenAPI docs	Moderate – more verbose, but simple deployment
Performance	Good for I/O-bound workloads (event loop)	Good; excellent for data/ML-heavy endpoints via native Python libraries	Excellent – compiled, very low latency, high concurrency
AI/ML integration	Calls out to Python/TF services; not a native ML runtime	Best – native access to TensorFlow/PyTorch/scikit-learn	Requires bridging to Python/ML services
Real-time capability	Excellent – native WebSocket/Socket.io support, pairs naturally with Firebase	Good, via WebSockets/ASGI	Excellent – lightweight goroutines suit high-concurrency sockets
Ecosystem/hiring	Very large talent pool, shared skillset with frontend team	Large and fast-growing, strong in data science circles	Smaller pool, steeper learning curve for a mid-sized team
Maintainability	High with NestJS's opinionated structure	High, type hints + FastAPI's structure aid maintainability	High once idiomatic Go patterns are established

Table 2.1 – Backend framework comparison

2.2 Database Options

Criterion	PostgreSQL	MongoDB	Firebase (Firestore/RTDB)	DynamoDB
Data model fit	Relational – ideal for structured user/health records requiring integrity constraints	Document – flexible for evolving feature sets	Document + realtime sync – ideal for social feeds/presence	Key-value/wide-column – very high scale, simple access patterns

Criterion	PostgreSQL	MongoDB	Firebase (Firestore/RTDB)	DynamoDB
Scalability	Vertically strong, horizontally scalable with read replicas/partitioning	Scales horizontally with sharding	Auto-scales, managed by Google	Virtually unlimited, fully managed by AWS
Query performance	Excellent for complex relational/analytical queries (joins, aggregates)	Good for document lookups; joins are costly	Good for simple queries; complex analytics need export to BigQuery	Excellent for known access patterns; complex ad-hoc queries are limited
Health data handling	Strong fit – ACID transactions, mature encryption/row-level security for sensitive fields	Workable, needs careful schema/validation discipline	Workable for non-sensitive social data; sensitive health data best kept elsewhere	Workable but access-pattern modelling adds complexity for health records
Real-time capability	Requires add-ons (e.g. LISTEN/NOTIFY, external services)	Change streams available	Native, best-in-class real-time sync	Streams available via DynamoDB Streams
Cost (mid-size team)	Low to moderate, many managed options (RDS, Supabase, Neon)	Moderate	Low to start, can rise with heavy read/write volume	Pay-per-request can be economical but harder to predict

Table 2.2 – Database comparison

2.3 Authentication and Authorization

Criterion	Firebase Auth	AWS Cognito	Auth0	Supabase Auth
Setup speed	Very fast, tightly integrated with other Firebase services already used for real-time/social	Moderate – more configuration, deep AWS ecosystem fit	Fast, polished dashboard and SDKs	Fast, integrates directly with a Postgres backend
Compliance (GDPR/CCPA/HIPAA-adjacent)	GDPR/CCPA compliant with proper configuration; no native HIPAA BAA	Strong compliance posture, HIPAA-eligible under a BAA	Strong compliance posture, enterprise-grade, HIPAA add-on available	GDPR-ready; compliance maturity still developing
Social/federated login	Excellent, very broad provider support	Good, broad provider support	Excellent, broadest enterprise SSO/social support	Good, growing provider support
Cost (mid-size team)	Low, generous free tier	Low, pay-as-you-go, can be complex to estimate	Higher at scale (per-active-user pricing)	Low, bundled with Supabase's Postgres pricing
Maintainability	Very low overhead, managed entirely by Google	Moderate – more IAM/config surface area	Low overhead, but adds a third-party dependency distinct from the rest of the stack	Low overhead if already using Supabase/Postgres

Table 2.3 – Authentication provider comparison

2.4 Recommended Combination

Recommended combination: Node.js/Express backend + PostgreSQL (sensitive, structured health/user data) + Firebase Firestore/Realtime Database (social feeds, presence, notifications) + Firebase Authentication.

Justification: Node.js/Express keeps the backend team working in the same language as the React Native frontend, reducing context-switching and onboarding time for a mid-sized team. PostgreSQL is retained for anything requiring strong data integrity and auditability – workout history, personal health metrics, and consent records – which strengthens GDPR/CCPA accountability. Firebase is layered in specifically for its unmatched real-time capability (social feed updates, live challenge leaderboards, push notifications), which would be expensive to replicate on PostgreSQL alone. Firebase Auth is chosen over Cognito/Auth0 because the team already depends on Firebase for real-time data, minimizing the number of distinct vendor integrations while still meeting FitFlow's privacy requirements when configured with strict security rules and field-level encryption for any health-adjacent data mirrored into Firestore.

Activity 3: Technology Comparison Matrix

The individual comparisons above are consolidated below into weighted decision matrices. Each option is scored 1 (poor) to 5 (excellent) against FitFlow-specific criteria; each criterion is weighted according to its importance to the project, and a weighted total is calculated as $\Sigma(\text{score} \times \text{weight})$. Weights are expressed as a fraction of 1.0 and reflect FitFlow's priorities: speed to market and cost-effectiveness (start-up context), real-time social features (core differentiator), AI/ML support (flagship feature), and strong security/compliance (health data).

3.1 Frontend Weighted Matrix

Criterion (weight)	Flutter	React Native	Kotlin MP	Swift/SwiftUI
Development speed (0.20)	4 = 0.80	5 = 1.00	3 = 0.60	2 = 0.40
Code reusability (0.15)	5 = 0.75	5 = 0.75	3 = 0.45	1 = 0.15
Performance (0.15)	4 = 0.60	4 = 0.60	5 = 0.75	5 = 0.75
Ecosystem/hiring (0.15)	3 = 0.45	5 = 0.75	3 = 0.45	3 = 0.45
Web compatibility (0.10)	4 = 0.40	4 = 0.40	2 = 0.20	1 = 0.10
AI/ML integration (0.15)	4 = 0.60	4 = 0.60	4 = 0.60	4 = 0.60
Maintenance cost (0.10)	4 = 0.40	4 = 0.40	3 = 0.30	2 = 0.20
Weighted Total (1.00)	4.00	4.50	3.35	2.65

Table 3.1 – Frontend weighted decision matrix (React Native ranks highest)

3.2 Backend Weighted Matrix

Criterion (weight)	Node.js/Express	Python/FastAPI	Go
Development speed (0.25)	5 = 1.25	4 = 1.00	3 = 0.75
Real-time capability (0.20)	5 = 1.00	3 = 0.60	5 = 1.00
AI/ML integration (0.20)	3 = 0.60	5 = 1.00	2 = 0.40
Ecosystem/hiring (0.15)	5 = 0.75	4 = 0.60	2 = 0.30
Maintainability (0.10)	4 = 0.40	4 = 0.40	4 = 0.40
Performance (0.10)	3 = 0.30	3 = 0.30	5 = 0.50
Weighted Total (1.00)	4.30	3.90	3.35

Table 3.2 – Backend weighted decision matrix (Node.js/Express ranks highest)

3.3 Database Weighted Matrix

Criterion (weight)	PostgreSQL	MongoDB	Firebase	DynamoDB
Health data handling (0.30)	5 = 1.50	3 = 0.90	2 = 0.60	3 = 0.90
Real-time capability (0.20)	2 = 0.40	3 = 0.60	5 = 1.00	3 = 0.60
Scalability (0.20)	4 = 0.80	4 = 0.80	5 = 1.00	5 = 1.00
Query performance (0.15)	5 = 0.75	3 = 0.45	3 = 0.45	3 = 0.45
Cost (0.15)	4 = 0.60	4 = 0.60	4 = 0.60	3 = 0.45
Weighted Total (1.00)	4.05	3.35	3.65	3.40

Table 3.3 – Database weighted decision matrix (PostgreSQL ranks highest overall; Firebase selected as a complementary real-time store)

3.4 Authentication Weighted Matrix

Criterion (weight)	Firebase Auth	AWS Cognito	Auth0	Supabase Auth
Setup speed / integration fit (0.30)	5 = 1.50	3 = 0.90	4 = 1.20	4 = 1.20
Compliance (0.30)	3 = 0.90	5 = 1.50	5 = 1.50	3 = 0.90
Cost (0.20)	5 = 1.00	4 = 0.80	2 = 0.40	4 = 0.80
Maintainability (0.20)	5 = 1.00	3 = 0.60	4 = 0.80	4 = 0.80
Weighted Total (1.00)	4.40	3.80	3.90	3.70

Table 3.4 – Authentication weighted decision matrix (Firebase Auth ranks highest given existing Firebase dependency)

3.5 Consolidated Recommended Stack

Layer	Recommended Technology	Weighted Score
Frontend	React Native (+ React Native Web)	4.50 / 5
Backend	Node.js with Express (NestJS structure optional)	4.30 / 5
Primary database	PostgreSQL	4.05 / 5
Real-time/social store	Firebase Firestore / Realtime Database	3.65 / 5
Authentication	Firebase Authentication	4.40 / 5
AI/ML runtime	TensorFlow Lite (on-device) + cloud inference service	–
Caching	Redis	–

Table 3.5 – Final consolidated technology stack for FitFlow

This consolidated stack mirrors and formalizes the choices already made informally in the case study (React Native, Node.js/Express, Firebase), while adding PostgreSQL as the system of record for sensitive structured health data and Redis for performance-critical caching – both of which strengthen the non-functional requirements for security, performance, and offline resilience identified during the requirements phase.

Activity 4: High-Level Architecture

4.1 Architecture Diagram

FitFlow - High-Level System Architecture

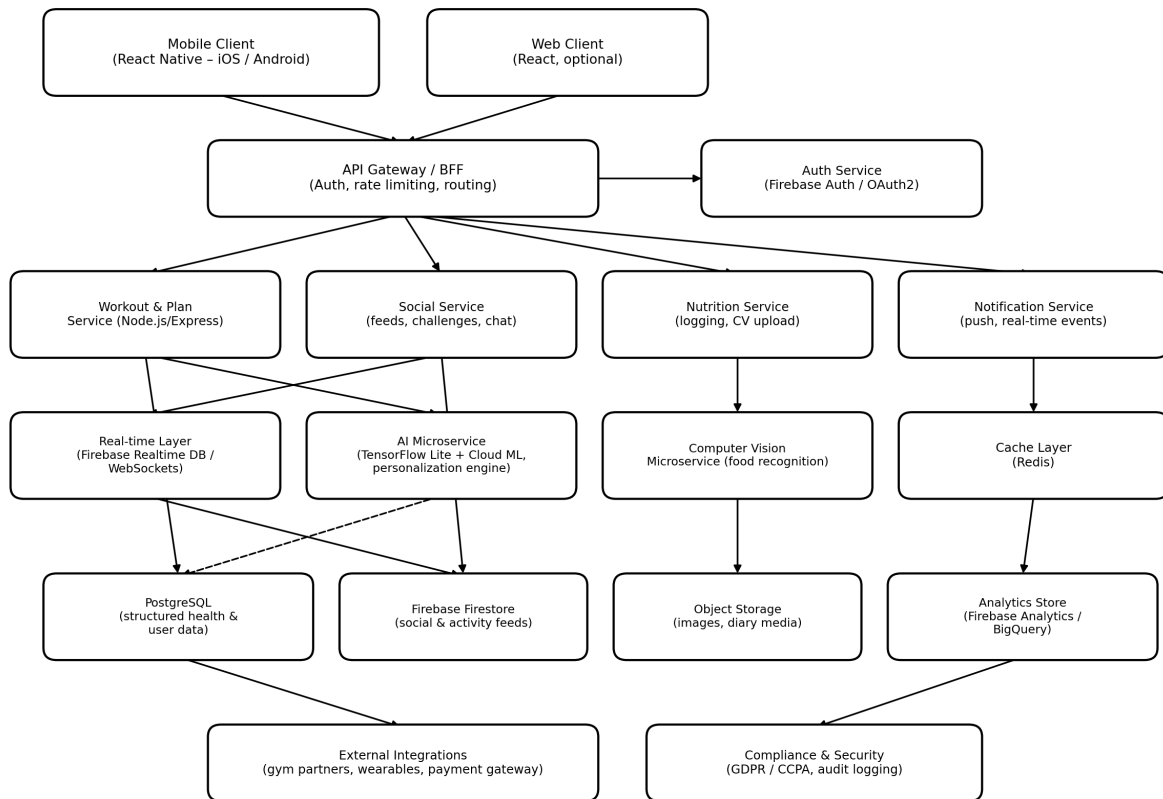


Figure 4.1 – FitFlow high-level system architecture (client, gateway, services, AI/CV microservices, data and compliance layers)

4.2 Key Components

- Mobile/Web Client (React Native): renders the home dashboard, AI “Daily Flow” card, workout builder, social feed and nutrition logger validated during usability testing.
- API Gateway / BFF: single entry point handling authentication checks, rate limiting and request routing to the correct microservice.
- Auth Service (Firebase Auth): issues and validates tokens for all client requests.
- Workout & Plan Service: owns workout templates, adaptive plan logic, and calls the AI Microservice for personalization.
- Social Service: manages private circles, challenges and group chat, backed by the real-time layer.
- Nutrition Service: receives food photos and structured meal logs, delegating recognition to the Computer Vision microservice.
- Notification Service: fan-out of push and in-app real-time events (streak reminders, celebratory badges).
- AI Microservice: hosts the on-device TensorFlow Lite model plus a cloud fallback for heavier personalization inference.
- Computer Vision Microservice: food-image recognition pipeline for nutrition logging.
- Real-time Layer (Firebase): powers live social feed updates and presence.
- Cache Layer (Redis): reduces latency for frequently accessed dashboard and leaderboard data.
- Data layer: PostgreSQL (structured/health data), Firestore (social/activity feeds), object storage (media), and an analytics store for cohort/retention analysis.

- External Integrations & Compliance: gym partner APIs, wearable integrations, and a dedicated compliance/audit-logging layer for GDPR/CCPA.

4.3 Data Flow Examples

Personalized workout plan

Client requests a plan → API Gateway authenticates the request → Workout & Plan Service reads user history from PostgreSQL → calls the AI Microservice, which blends the on-device model output with cloud inference → the resulting adaptive plan is returned to the client and cached in Redis for fast repeat access.

Social sharing

Client posts an update or challenge result → API Gateway routes to the Social Service → the event is written to Firestore and pushed through the real-time layer to friends' feeds → the Notification Service fans out push alerts to relevant circle members.

Nutrition tracking

Client uploads a meal photo → API Gateway routes to the Nutrition Service → the image is sent to the Computer Vision microservice for food recognition → the resulting structured entry is stored in PostgreSQL, the photo itself in object storage, and a summary event is emitted to the analytics store for retention tracking.

4.4 Security, Scalability and Integration Considerations

- Security: all traffic over TLS; field-level encryption for health-adjacent fields in PostgreSQL; Firebase security rules restrict Firestore access to circle members only; consent and audit logs retained per GDPR/CCPA.
- Scalability: stateless services behind the API Gateway can scale horizontally; PostgreSQL read replicas absorb reporting/analytics load; Redis reduces repeated load on the AI and Workout services; Firebase scales automatically for real-time/social spikes (e.g. viral challenges).
- Integration: gym partner and wearable integrations are isolated behind the External Integrations boundary so third-party outages cannot affect core app functionality; the AI and Computer Vision microservices are independently deployable/scalable from the rest of the backend.
- Offline support: the mobile client caches the latest plan and recent logs locally so workouts and meal logging keep working without connectivity, syncing to PostgreSQL/Firestore once back online.

4.5 Architecture Decision Record (ADR)

Field	Detail
ADR ID / Title	ADR-001: Hybrid PostgreSQL + Firebase data layer with Node.js/Express and React Native
Status	Accepted
Context	FitFlow needs strong data integrity and auditability for sensitive health data, near-instant real-time updates for social features, rapid delivery from a mid-sized team, and a single-codebase mobile client covering iOS, Android and (optionally) web.
Decision	Adopt React Native for the client, Node.js/Express for backend services, PostgreSQL as the system of record for structured/health data, Firebase (Firestore/Realtime Database + Auth) for real-time social features and authentication, TensorFlow Lite with a cloud fallback for AI personalization, and Redis for caching.
Alternatives considered	Flutter (frontend); Python/FastAPI or Go (backend); MongoDB or DynamoDB as the sole database; AWS Cognito or Auth0 for authentication – rejected primarily on ecosystem fit, team-language consistency, or added vendor complexity relative to the marginal benefit for FitFlow's scale.

Field	Detail
Consequences (positive)	Faster delivery via a shared JavaScript/TypeScript skillset across frontend and backend; best-in-class real-time behaviour for social features; strong auditability for health data in PostgreSQL; lower authentication overhead by reusing Firebase already required for real-time sync.
Consequences (trade-offs)	Running two data stores (PostgreSQL and Firestore) adds operational complexity and requires clear rules on which data lives where; the team must maintain consistency between the two stores for any data that logically spans both (e.g. a user profile referenced by both health records and social features).

Table 4.1 – Architecture Decision Record for the FitFlow technology stack

Activity 5: GitHub Repository

5.1 Repository Setup

A new public/private GitHub repository named `fitflow-redesign` is created to host all supporting documents and code produced across Activities 1–4, using the structure and steps below.

Folder Structure

```

fitflow-redesign/
├── frontend/           # React Native app (iOS/Android/Web)
├── backend/            # Node.js/Express services (workout, social, nutrition, notifications)
├── ai-service/         # TensorFlow Lite + cloud personalization microservice
├── cv-service/         # Computer vision food-recognition microservice
├── docs/
│   ├── tech-stack-summary.md
│   ├── comparison-matrix.md
│   ├── architecture-diagram.png
│   └── adr/ADR-001-technology-stack.md
├── .github/workflows/ci.yml # CI pipeline
├── .gitignore
└── README.md

```

Repository Setup Steps

- Create the repository on GitHub named `fitflow-redesign` (Settings: default branch `main`, Add a README, add a Node `.gitignore` template).
- Enable branch protection on `main`: require a pull request before merging, require status checks (CI) to pass, and require at least one review.
- Clone locally: `git clone https://github.com/<org>/fitflow-redesign.git`
- Create the folder structure shown above and copy in the supporting documents from Activities 1–4 (comparison tables, weighted matrix, architecture diagram, ADR).
- Add a basic CI workflow (`.github/workflows/ci.yml`) to lint and test the frontend and backend on every push/pull request.
- Stage and commit: `git add . && git commit -m "Initial commit: FitFlow redesign tech stack, comparison matrix, architecture and ADR"`
- Push to GitHub: `git push origin main`

README.md (summary content)

The README documents the project purpose, the recommended technology stack (Table 3.5), a link to the architecture diagram (Figure 4.1) and ADR (Table 4.1), and instructions for running the frontend, backend, AI and CV services locally, so any new contributor can orient themselves without re-reading the full lab report.

5.2 Repository Link

Repository: <https://github.com/IT23551466/fitflow-redesign>

Note: replace the placeholder URL above with the actual repository link once the repository has been created under your own GitHub account, and ensure the first commit includes all documents listed in Section 5.1.

Conclusion

Building on FitFlow's validated user research and design work, this report selects React Native, Node.js/Express, a hybrid PostgreSQL/Firebase data layer, and Firebase Authentication as the technology stack best suited to the project's speed, cost, real-time and compliance requirements. The accompanying high-level architecture and ADR translate that decision into a concrete system design, and the proposed GitHub repository structure ensures all findings are version-controlled and ready for implementation.